Proceedings of Machine Learning Research vol 284:1–24, 2026 20th Conference on Neurosymbolic Learning and Reasoning

SymAlign-UAST: Neuro-Symbolic Code Understanding via Hyperbolic Geometry and Structural Invariance

Kwangje Baeg KWANGJE.BAEG@KT.COM
Korea Telecom Research

Hoyeob Kim HKIM3217@GATECH.EDU
Georgia Institute of Technology

Janaina Villacis JANA.VILLACIS@KDIS.AC.KR
KDI School of Public Policy and Management

Editors: Alessandra Mileo, Andrea Passerini and Cogan Shimizu

Abstract

Code understanding systems traditionally rely on identifier names, which are vulnerable to obfuscation and lack semantic depth. We present SymAlign-UAST, a neuro-symbolic framework for robust, interpretable code domain classification that integrates structural graph encoding via Graph Attention Networks on Universal Abstract Syntax Trees (UASTs), hyperbolic contrastive learning for hierarchical domain representations, and differentiable symbolic logic constraints. Our empirical results establish that code structure alone—independent of identifier naming—carries substantial semantic information. Pure structural encoding maintains stable classification accuracy under full identifier stripping with perfect obfuscation invariance, while surface-form baselines collapse by 79–86% relative under the same conditions. Hyperbolic geometry in Poincaré ball space organizes the retrieval space more hierarchically than flat Euclidean alignment, inducing stronger geometric class separation and a measurable norm-depth correlation aligned with the domain taxonomy. This hierarchical benefit comes at a cost to in-distribution accuracy, a trade-off between hierarchical regularization and closed-set memorization that we characterize and quantify. Collectively, these findings demonstrate a framework that is obfuscation-robust, hierarchically generalizable, and reliability-calibrated through knowledge-guided differentiable regularization. These properties are achievable in identifier-poor settings where surface-form models are insufficient, establishing structural encoding as a viable alternative wherever identifiers are absent or unreliable, such as compiled binaries, minified code, and cross-language transfer settings. The code and sample datasets are publicly available at: https://github.com/KwangjeBaeg/symalign-uast.

1. Introduction and Related Work

Code understanding has emerged as a central research topic within source code representation learning and AI-driven software engineering (Allamanis et al., 2018a; Niu et al., 2022), underpinning applications from intelligent code search to automated vulnerability detection. A fundamental tension, however, persists in how these systems represent programs: most rely on textual surface forms—identifier names, library paths, and import aliases—that are unavailable or semantically degraded in many real deployment settings. Models such as CodeBERT (Feng et al., 2020) and TF-IDF classifiers achieve strong performance by exploiting identifier names, yet fail when those names are obfuscated, minified, or simply absent. Consider two Python functions implementing identical numerical computation:

© 2026 K. Baeg, H. Kim & J. Villacis.

one with meaningful names (`import numpy as np; def compute_stats(data): ...`), the other obfuscated (`import a1b2c3 as x; def f(y): ...`). Current models correctly classify the first by recognizing `"numpy"`, but collapse on the second despite its identical computational semantics—exposing a critical dependence on surface-level naming rather than structural intent.

We study primary code domain classification under identifier-poor conditions: given a code snippet whose identifiers are stripped or obfuscated, predict its functional category (e.g., *Security*, *Networking*, *Data Science*) from structural features alone. This task matters whenever identifier-based signals are unavailable or unreliable, including decompiled-binary triage, coarse characterization of obfuscated production code, and analyst routing pipelines where library names and variable identifiers cannot be trusted. One concrete example arises in malware triage: an analyst receiving a stripped binary may need an early functional characterization before deep reverse engineering or symbol recovery, and structural topology alone can still provide enough signal to route the sample to the appropriate analysis queue. In all such settings, identifier-based models provide no effective classification signal, while the structural encoding—the pattern of control flow, call structure, and module organization—remains intact and domain-discriminative.

1.1. Background and Positioning

Our central hypothesis is that code structure alone, stripped of all identifier information, retains sufficient semantic signal for coarse domain prediction. Pre-trained code language models such as CodeBERT (Feng et al., 2020) and GraphCodeBERT (Guo et al., 2021) excel on identifier-rich code but are fundamentally dependent on surface-level naming; structure-aware models, including graph-based encodings (Allamanis et al., 2018b; Li et al., 2016) and path-based AST representations (Alon et al., 2019), encode structural relationships alongside leaf-node text, yet still exhibit 40–60% performance degradation when variable names are removed (Hellendoorn et al., 2020). We address this identifier-dependence gap with **SymAlign-UAST**: we parse each file into a Universal Abstract Syntax Tree (UAST) (Babelfish, 2017)—a language-agnostic intermediate representation—and train a Graph Attention Network (GAT) (Veličković et al., 2018) exclusively on identifier-stripped graph topology, guaranteeing that any identifier-renaming transformation leaves predictions unchanged. To our knowledge, no prior work has systematically evaluated graph neural network (GNN) classification on fully identifier-stripped UAST topology.

Structural encoding alone is insufficient for generalization, because software domain taxonomies are inherently hierarchical (e.g., *Security* $\subset$ *System Programming*) and Euclidean spaces distort tree-structured hierarchies (Krioukov et al., 2010). Poincaré embeddings (Nickel and Kiela, 2017; Ganea et al., 2018) show that **hyperbolic space** encodes such structure with lower distortion. We therefore project embeddings into the Poincaré ball for hierarchy-aware contrastive alignment and zero-shot prototype retrieval over the domain taxonomy.

Geometric alignment alone does not prevent logically inconsistent predictions. Neuro-symbolic methods (d'Avila Garcez et al., 2019; Serafini and d'Avila Garcez, 2016; Xu et al., 2018) show that differentiable symbolic constraints can be incorporated into neural training objectives. Prior work spans loss-based penalties, constraint-layer architectures, and

post-hoc corrections (Giunchiglia et al., 2022). We adopt the loss-based family, formulating **knowledge-guided differentiable regularization** as fuzzy-logic-inspired penalty terms (Hájek, 1998; Van Krieken et al., 2022) over structural integrity, behavioral consistency, and domain exclusivity. This integrates into the GNN–hyperbolic pipeline while avoiding the accuracy cost of hard post-hoc corrections.

1.2. Contributions

SymAlign-UAST addresses three distinct failure modes of prior approaches:

- **Structural encoding via GAT on UAST**: enables identifier-invariant domain prediction by operating solely on identifier-stripped program structure.
- **Hyperbolic contrastive alignment**: organizes code and domain representations in a hierarchy-aware retrieval space for zero-shot transfer.
- **Differentiable symbolic regularization**: injects domain knowledge during training to encourage more logically consistent and reliability-aware predictions.

These components define a framework for code understanding that is obfuscation-robust, hierarchy-aware, and knowledge-guided under identifier-poor conditions.

2. Method

2.1. Problem Formulation

Task. Given a code snippet c, we predict its primary domain $y \in \mathcal{Y}$, where $\mathcal{Y}$ is a set of 14 distinct domains:

$$\mathcal{Y} = \{\text{Web, Data Science, ML/AI, Science, System, Security, } \ldots, \text{ DevOps}\}.$$

Input Representation. We parse c into a UAST and construct a directed graph $\mathcal{G} = (V, E)$, where nodes V represent syntax elements (e.g., `FunctionDef`, `Call`) and edges E encode structural relationships. Identifier strings—variable names, function names, and import aliases—are stripped prior to encoding, retaining only syntactic type information; the GNN thus observes only node types and graph topology. Appendix E provides three full-length pre/post UAST examples covering function-, module-, and class-level cases.

Objective. The goal is a predictor invariant to identifier renaming—that is, renaming any variable, function, or import alias should not change the predicted domain—guaranteed by construction since all identifier strings are stripped before encoding and the GNN observes only node types and graph topology. The architecture comprises three modules: a structural encoder that maps $\mathcal{G}$ to a Euclidean embedding $\mathbf{z}_E \in \mathbb{R}^d$, a hyperbolic projector that maps $\mathbf{z}_E$ into the Poincaré ball $\mathbb{B}_c^d$ to capture hierarchical domain relationships, and a neuro-symbolic classifier that predicts domain probabilities under logic-inspired differentiable penalty terms.

2.2. Structural Encoder: GAT on UAST

We employ a 3-layer Graph Attention Network (GAT) (Veličković et al., 2018) to encode UAST structure. Each node $v_i \in V$ is initialized with a k-dimensional one-hot node-type vector $\mathbf{x}_i^{(0)} \in \mathbb{R}^k$ corresponding to its syntactic type, such as `If`, `For`, or `Call`. The string-based `code_snippet` field is excluded entirely from model input; the GNN processes only graph topology and these discrete node-type features. At each layer, nodes aggregate information from their neighbors via multi-head attention coefficients $\alpha_{ij}^{(\ell)}$. Residual connections are applied between layers. The final graph representation $\mathbf{z}_E \in \mathbb{R}^d$ is obtained via global mean pooling over all node embeddings at the final layer. The 3-layer architecture was selected via validation search over depths in $\{2, 3, 4\}$; the full sensitivity analysis is reported in Appendix B.

2.3. Hyperbolic Contrastive Alignment

Software domain taxonomies exhibit latent hierarchies (e.g., *System* $\supset$ *Security*), which Euclidean spaces represent with distortion. We decouple structural encoding from hierarchical organization: the GAT encoder extracts local structural features into $\mathbf{z}_E$, while hyperbolic alignment in the Poincaré ball imposes hierarchy-aware geometry over domain prototypes. The Euclidean encoder is not designed to represent hierarchical domain geometry directly. Instead, it captures local structural regularities, while the hyperbolic alignment stage explicitly imposes tree-structure-consistent organization in the retrieval space. This allows hierarchy to be imposed in retrieval space without burdening the encoder with inter-domain geometry, enabling zero-shot transfer to unseen domains via prototype proximity.

We project $\mathbf{z}_E$ to the Poincaré ball $\mathbb{B}_c^d = \{\mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\| < 1/\sqrt{c}\}$ using the exponential map at the origin:

$$\mathbf{z}_H = \exp_{\mathbf{0}}^c(\mathbf{z}_E) = \tanh\left(\sqrt{c}\|\mathbf{z}_E\|\right) \frac{\mathbf{z}_E}{\sqrt{c}\|\mathbf{z}_E\|}.$$

This maps $\mathbf{z}_E$ onto the Poincaré ball, preserving the direction of the structural embedding while contracting its norm into the open ball $\mathbb{B}_c^d$. The hyperbolic distance between two points $\mathbf{u}, \mathbf{v} \in \mathbb{B}_c^d$ is given by:

$$d_{\mathbb{B}}(\mathbf{u}, \mathbf{v}) = \frac{2}{\sqrt{c}} \operatorname{arctanh}\left(\sqrt{c}\| - \mathbf{u} \oplus_c \mathbf{v}\|\right),$$

where $\oplus_c$ denotes Möbius addition. Curvature $c = 1.0$ was selected by validation zero-shot performance; Appendix C reports the full sweep. Riemannian gradient descent is performed using the `geoopt` library (Kochurov et al., 2020), a PyTorch package for Riemannian optimization on smooth manifolds.

Domain Prototypes and Zero-Shot Inference. To enable classification over both seen and unseen domains without a fixed classifier head, we construct a prototype $\mathbf{p}_d$ for every domain $d \in \mathcal{D}$ from a short textual description (e.g., *"computer security, cryptography, authentication"* for *Security*) encoded by a frozen `all-mpnet-base-v2` (Reimers and Gurevych, 2019) sentence-transformer into a 768-dimensional semantic vector $\mathbf{e}_d$. A shared two-layer projection ϕ_s (Linear–ReLU–Linear), trained jointly with the graph encoder, maps $\mathbf{e}_d$ into the Poincaré ball:

$$\mathbf{p}_d = \exp_{\mathbf{0}}^c\big(\phi_s(\mathbf{e}_d)\big).$$

This embeds each domain prototype directly in the Poincaré ball via the same exponential map, making hyperbolic distance a unified retrieval metric across all domains. Because ϕ_s is shared across all domains, unseen domains are projected into the same hyperbolic space without any labeled examples. At inference time, zero-shot prediction selects the nearest prototype by hyperbolic distance: $\hat{y}_{ZS} = \arg\min_{d \in \mathcal{D}} d_{\mathbb{B}}(\mathbf{z}_H, \mathbf{p}_d)$, where $\mathcal{D}$ includes both seen and unseen domains. In-distribution predictions use the cross-entropy (CE) classifier head trained on seen domains, evaluated by Top-1 accuracy and mean reciprocal rank (MRR). Zero-shot predictions use hyperbolic distance-based retrieval exclusively, with the CE head discarded.

Contrastive Alignment Objective. Training optimizes a hyperbolic contrastive loss that pulls code embeddings toward their ground-truth domain prototype and pushes them away from negatives. For a mini-batch of N code–domain pairs $\{(\mathbf{z}_H^{(i)}, \mathbf{p}_{y_i})\}_{i=1}^{N}$:

$$\mathcal{L}_{contrastive} = -\frac{1}{N} \sum_{i=1}^{N} \log \frac{\exp\big(-d_{\mathbb{B}}(\mathbf{z}_H^{(i)}, \mathbf{p}_{y_i})/\tau\big)}{\sum_{d \in \mathcal{D}} \exp\big(-d_{\mathbb{B}}(\mathbf{z}_H^{(i)}, \mathbf{p}_d)/\tau\big)},$$

where $\tau = 0.1$ is the temperature, $d_{\mathbb{B}}$ is the Poincaré distance, and all domain prototypes $\mathbf{p}_d$ serve as negatives for samples not in domain d. This $\mathcal{L}_{contrastive}$ appears in the total objective in Section 2.4.

2.4. Knowledge-Guided Differentiable Regularization

Following the loss-based family of constraint integration (Giunchiglia et al., 2022), we encode domain knowledge as differentiable penalty terms rather than hard architectural projections, post-hoc corrections, or discrete symbolic entailment—favoring soft, training-time calibration over brittle deterministic enforcement at inference time. The fuzzy relaxations are selected to match the semantics of the corresponding constraints. Łukasiewicz-style implications are used for rule-like constraints. For exclusivity, a product T-norm is used because it directly penalizes simultaneous high confidence on mutually incompatible domains. This operator-specific design is consistent with prior work showing that the behavior of differentiable fuzzy constraints depends critically on the chosen logic operators (Van Krieken et al., 2022).

We augment the neural model with knowledge-guided differentiable regularization by defining three logic-inspired penalty terms over domain constraints, relaxed into differentiable loss functions using fuzzy logic semantics. Specifically, structural integrity and behavioral consistency rules are relaxed via Łukasiewicz-style implications, while the domain exclusivity constraint employs a product T-norm penalty consistent with the continuous conjunction of two predicted probabilities. Let $\mathbf{p} \in [0,1]^{|\mathcal{Y}|}$ denote the predicted probability distribution over domains. We write the predicate $P(y)$ for "domain y is predicted," grounded by its soft truth value p_y, the y-th component of $\mathbf{p}$.

2.4.1. Symbolic Evidence Extraction

We extract two evidence vectors from the raw UAST, using $\mathbf{s}$ notation distinct from the text embedding $\mathbf{e}_d$. **Structural evidence** $\mathbf{s}_{struct} \in [0,1]^K$ captures node-type proportions (function, class, call, import, and comment ratios), call-to-function ratio, and average branching

factor; mean($\mathbf{s}_{struct}$) serves as a structural complexity score in $\mathcal{L}_{struct}$. **Behavioral evidence** $\mathbf{s}_{behav} \in \{0,1\}^7$ contains binary flags for seven abstract behaviors matched by a task-specific manually curated rule set over `uast_package` import prefixes and `uast_call` signatures; Appendix E.1 lists representative mappings. $\mathbf{M}_y \in \{0,1\}^7$ is a domain-behavior compatibility mask initialized from LLM-assisted ontology drafting and finalized by author curation; Figure 3b visualizes the matrix and Appendix E.1 provides exact binary entries.

2.4.2. Differentiable Logic Losses

Structural Integrity ($\mathcal{L}_{struct}$). We relax $\forall y : P(y) \implies \text{HasStructure}(c)$ to:

$$\mathcal{L}_{struct} = \mathbb{E}[\max(0, \max(\mathbf{p}) - \text{mean}(\mathbf{s}_{struct}))].$$

Intuitively, this penalizes high-confidence predictions on structurally thin inputs: if the model is certain about a domain assignment yet the code exhibits minimal structural complexity, the loss pushes confidence downward, encouraging calibrated uncertainty.

Behavioral Consistency ($\mathcal{L}_{behav}$). We relax $\forall y : P(y) \implies \exists b\text{: } (\text{HasBehavior}(b) \wedge \text{Compatible}(y, b))$ to:

$$\mathcal{L}_{behav} = \mathbb{E}\left[\sum_{y \in \mathcal{Y}} \max\left(0, p_y - \frac{\mathbf{s}_{behav} \cdot \mathbf{M}_y}{\|\mathbf{M}_y\|_1 + \epsilon}\right)\right],$$

where $\mathbf{M}_y$ is the manually curated domain-behavior compatibility mask. Intuitively, this penalizes predicting domain y with confidence exceeding the normalized behavioral support for y: a high-confidence prediction of *Networking* is penalized when no network-related behavior flags are active in the UAST.

Domain Exclusion ($\mathcal{L}_{excl}$). We treat domain pairs as mutually exclusive when their empirical co-occurrence falls below 1%; Appendix A lists all such pairs. These are constrained via $\forall (y_i, y_j) \in \mathcal{R}_{excl} : \neg(P(y_i) \wedge P(y_j))$, relaxed by product T-norm:

$$\mathcal{L}_{excl} = \sum_{(y_i, y_j) \in \mathcal{R}_{excl}} p_{y_i} \cdot p_{y_j}.$$

Intuitively, this product is large only when both p_{y_i} and p_{y_j} are simultaneously high, directly penalizing logically inconsistent co-predictions over mutually exclusive domain pairs.

2.4.3. Total Objective

The final loss balances classification accuracy, geometric alignment, and logical consistency:

$$\mathcal{L}_{total} = \mathcal{L}_{CE} + \lambda_{con}\mathcal{L}_{contrastive} + \lambda_{struct}\mathcal{L}_{struct} + \lambda_{behav}\mathcal{L}_{behav} + \lambda_{excl}\mathcal{L}_{excl},$$

where $\lambda_{con} = 0.5$ and the symbolic loss weights λ_{struct}, λ_{behav}, λ_{excl} are selected via grid search on the validation set. Appendix B provides optimal weight values, the full hyperparameter sensitivity analysis, and training implementation details including optimizer, learning rate, and hardware configuration.

3. Experiments

We report three experiments addressing three core hypotheses about identifier-poor code understanding. Section 3.2 evaluates whether structural encoding provides concrete robustness over modern pretrained code models under a spectrum of identifier-corruption conditions (H1: structural sufficiency). Section 3.3 assesses the effect of hyperbolic alignment on retrieval-space structure and zero-shot generalization via a controlled geometry ablation (H2: geometric structure). Section 3.4 analyzes the directional reliability signal, aggregate magnitude, and component contributions of symbolic regularization against shuffled-ontology controls, hard post-hoc correction, and component ablation findings (H3: symbolic guidance).

3.1. Dataset Construction and Preprocessing

We curated 7,615 Python files spanning 14 software domains from 247 open-source GitHub repositories, with Security and DevOps held out exclusively for zero-shot evaluation. All files are parsed into identifier-stripped UASTs via the Babelfish framework (Babelfish, 2017). Appendix E reports domain selection criteria, labeling validation, and dataset statistics.

We compare SymAlign-UAST against three primary baselines: TF-IDF with Logistic Regression (LR) trained on raw identifier names (93.62% clean accuracy, establishing a surface-form upper bound); a structure-only GNN trained on identifier-stripped UASTs (isolating pure structural topology); and a Euclidean Contrastive variant applying the same contrastive objective in flat $\mathbb{R}^n$ space (the geometric ablation baseline for Section 3.3). The robustness benchmark in Section 3.2 additionally includes GraphCodeBERT and CodeBERT; fine-tuning protocols and preprocessing details are provided in Appendix H.

3.2. Identity-Agnostic Robustness

We evaluate whether the identity-agnostic design of SymAlign-UAST provides a concrete robustness advantage relative to modern pretrained code models. Five models spanning surface-form, structure-only, and hybrid paradigms are evaluated across five obfuscation conditions (C0: clean; C4: all identifiers fully stripped; full protocol in Appendix H), trained on clean code and tested independently (5 seeds). *Retention ratio* (C4/C0) quantifies obfuscation-robustness as a model property; this benchmark targets robustness under identifier corruption rather than absolute performance.

Table 1: Top-1 accuracy (%) and obfuscation retention ratio across representation families. Results are averaged over five seeds. All models are trained on clean code and evaluated under each obfuscation condition. C1: import identifiers masked only; C4: full identifier stripping. Retention = C4/C0. Full 5-condition results are reported in Appendix H.

Model	Clean (C0)	Import-Masked (C1)	Full Obfusc. (C4)	Retention
GraphCodeBERT	**96.59**	19.21	13.89	0.144
TF-IDF + LR	93.62	12.63	12.63	0.135
GNN-only (structure)	47.79	47.79	47.79	**1.000**
SymAlign-UAST (ours)	31.91	31.91	31.91	**1.000**

Table 1 summarizes robustness across three representative conditions. Modern pretrained models dominate on clean code—GraphCodeBERT reaches 96.59% Top-1—but import masking alone (C1) triggers near-total collapse: GraphCodeBERT drops to 19.21% (retention 0.144) and TF-IDF+LR to 12.63% (retention 0.135). Under full obfuscation (C4), both identifier-agnostic models retain 100% by construction. A finer C1/C2 vulnerability decomposition and the full five-condition table are reported in Appendix H.

The resulting picture is one of *regime dependence*: surface-form models are preferable when identifiers are reliable, while structure-first models are uniquely robust when those signals collapse. SymAlign-UAST retains perfect obfuscation invariance (retention 1.000), though its clean supervised accuracy (31.91%) is lower than surface-form models. Section 3.3 analyzes this gap as a deliberate trade-off between hierarchical retrieval-space structuring and closed-set memorization.

3.3. Geometric Generalization

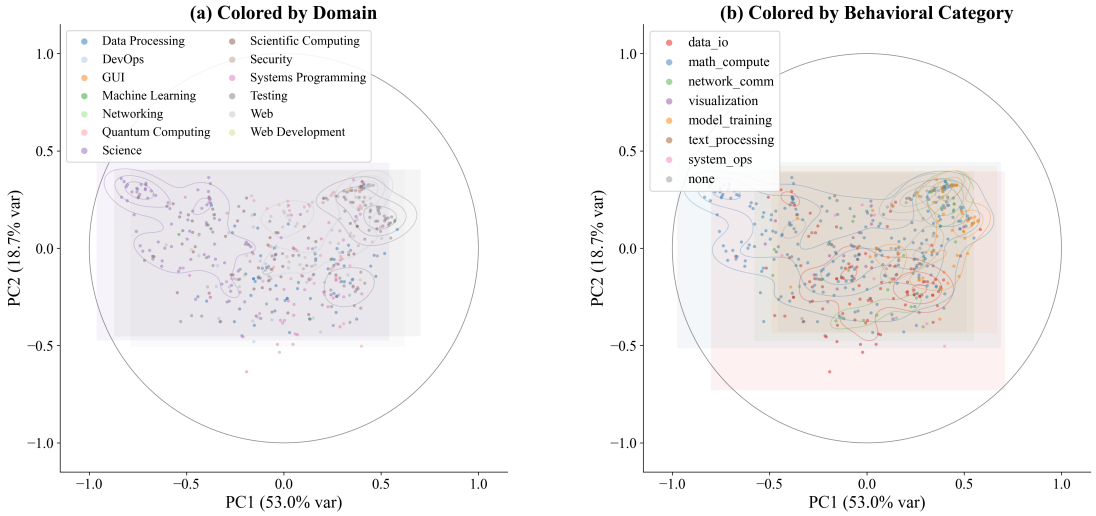


Figure 1: **Poincaré ball domain embeddings (E3, hybrid hyperbolic).** Broader domains (e.g. *Web*) lie near the origin, while more specialized domains (e.g. *Security*) move toward the boundary, consistent with the norm-depth correlation (ρ=−0.253; Table 3). Appendix G discusses the attractor-like pseudo-clusters under scale limitations.

Figure 1 visualizes the learned domain embeddings under E3. Tables 2–3 report a controlled geometry ablation in which E1–E3 share the same Euclidean GAT encoder and differ only in alignment space. E1 achieves the highest in-distribution Top-1 but zero-shot Top-1 is 0%, confirming that prototype alignment is necessary for unseen-domain transfer.

The central comparison is E2 vs. E3, which isolates the effect of switching the retrieval space from Euclidean to hyperbolic while holding the encoder fixed. Raw zero-shot retrieval is not significantly improved: ZS Top-1 is 19.0±1.8% (E3) vs. 17.9±2.5% (E2) and MAP@5 is essentially tied (0.303 vs. 0.301, both p>0.4). The geometric structure of the learned space is, however, consistently stronger under hyperbolic alignment: Table 3 shows E3 achieves a substantially higher class separation ratio (0.803 vs. 0.680, p=0.020, Cohen's d=1.69)

Table 2: Geometry ablation performance. Results are reported as mean±std over five seeds under balanced early stopping. E1–E3 share the Euclidean GAT encoder and differ only in alignment and retrieval space. E4 is an exploratory fully hyperbolic encoder variant included for reference.

Model	Retrieval	In-Dist Top-1	ZS Top-1	MAP@5	kNN-10	Conv
E1: CE-only	classifier	**42.9±1.3%**	0.0 ± 0.0%	0.000	0.000	5/5
E2: Eucl. Contr.	Euclidean	37.0±5.7%	17.9±2.5%	0.301±0.024	0.471±0.020	5/5
E3: Hybrid Hyp.	Hyperbolic	30.4±2.1%	**19.0±1.8%**	**0.303±0.011**	0.462±0.024	5/5
E4: Fully Hyp.	Hyperbolic	31.7±1.4%	11.0±5.1%	0.223±0.057	0.408±0.012	5/5

Table 3: Geometry diagnostics. Results are reported as mean±std over five seeds under balanced stopping. Sep. Ratio = $d_{\text{inter}}/d_{\text{intra}}$, where higher values indicate better-separated clusters. Norm-depth ρ denotes the Spearman correlation between embedding norm and domain behavioral generality. It is undefined for E2 because unit-sphere normalization fixes the norm. **Bold**: most hierarchy-consistent per column.

Model	Sep. Ratio	Silhouette	Norm-depth ρ	Stable
E1: CE-only	0.657±0.027	−0.184 ± 0.045	−0.379 ± 0.154	5/5
E2: Eucl. Contr.	0.680±0.028	−0.054 ± 0.009	—	5/5
E3: Hybrid Hyp.	**0.803±0.065**	−0.150 ± 0.047	**−0.253 ± 0.054**	5/5
E4: Fully Hyp.	0.783±0.047	−0.180 ± 0.030	−0.304 ± 0.109	5/5

and a measurable norm-depth correlation (ρ=−0.253, p<0.05) corroborated by Figure 1. This indicates that the benefit of hyperbolic alignment lies in geometry-aware hierarchical organization rather than raw retrieval gains. Crucially, E2 and E3 share an identical Euclidean encoder; the geometry difference arises solely from the retrieval space, confirming that hierarchical structure is imposed at the alignment stage rather than by the encoder. The in-distribution drop for E3 (30.4% vs. 37.0%, p=0.047) reflects a structural trade-off between hierarchical regularization and closed-set memorization. Stopping-criterion sensitivity and the exploratory fully hyperbolic variant (E4) are reported in Appendix G.

3.4. Symbolic Regularization Analysis

Figure 2 and Table 4 show that symbolic regularization is directionally meaningful but modest in aggregate. The Pearson correlation between prediction entropy and logic violation magnitude shifts from r=+0.04 (CE-only baseline, E1) to r=−0.33 (SymAlign), indicating that uncertainty concentrates on logically inconsistent predictions. Table 4 isolates the symbolic-specific contribution across five controlled variants. Soft regularization (S1) yields only modest aggregate change over the no-symbolic baseline (S0): violation rate decreases by 0.25pp and ECE by

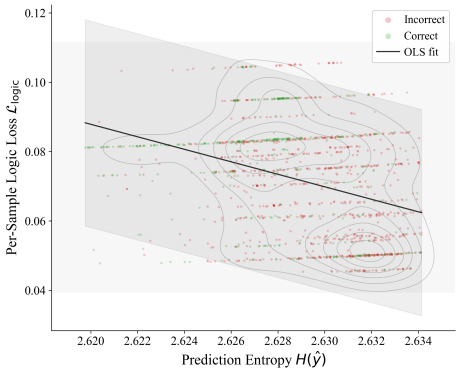


Figure 2: **Entropy vs. Logic Violation.** SymAlign (r=−0.33) vs. baseline (r=0.04).

0.11pp, and the shuffled-ontology control (S4) produces nearly indistinguishable behavior, indicating that at the current dataset scale the effect is partly generic. Hard post-hoc correction (S3) achieves the largest reliability gains (−8.7pp violation rate, −9.7pp ECE) at a −9.4pp Top-1 cost, supporting soft training-time integration when accuracy must be preserved. Appendix J further shows that $\mathcal{L}_{contrastive}$ is the prerequisite for zero-shot transfer and that $\mathcal{L}_{behav}$ alone causes the sharpest in-distribution instability. Appendix I provides additional symbolic controls.

Table 4: Symbolic regularization analysis with the E3 backbone. EntCorr denotes the Pearson correlation between prediction entropy and violation magnitude, where more negative values indicate stronger alignment.

Variant	Top-1	ECE	ViolRate	EntCorr	ZS Top-1
S0 No-Symbolic	35.51%	0.2628	0.2823	−0.345	19.33%
S1 Current-Sym	35.64%	0.2640	0.2799	−0.360	19.19%
S2 Łukasiewicz	35.19%	**0.2590**	**0.2792**	**−0.374**	18.44%
S3 Hard PostHoc	26.21%	0.1669	0.1929	−0.334	18.71%
S4 Shuffled-Ontol.	35.46%	0.2623	0.2854	−0.345	18.92%

4. Discussion

Section 3.2 establishes a regime dependence that implies two deployment contexts: structure-first models for identifier-poor settings, and surface-form models where identifiers remain reliable. Hyperbolic alignment's primary benefit lies in organizing the retrieval space consistently with domain hierarchy rather than producing large raw gains in zero-shot accuracy. At the current dataset scale, the symbolic component functions as a calibration-oriented regularizer, improving the directional alignment between uncertainty and logical inconsistency, but yields modest aggregate gains.

Three limitations motivate future work. The dataset is modest relative to large-scale pretraining corpora and the 14-category ontology is coarse-grained. Self-supervised pretraining on larger UAST collections would address both. Riemannian optimization incurs approximately 2× the training overhead of standard GNNs. Mixed-curvature spaces (Gu et al., 2019) offer a path toward reducing this cost.

5. Conclusion

We presented SymAlign-UAST, a neuro-symbolic framework that demonstrates code structure alone carries substantial semantic information for domain classification under identifier-poor and adversarial conditions. Our results indicate that structural encoding provides a viable foundation when identifier signals are unavailable or unreliable, while hyperbolic alignment and knowledge-guided regularization improve hierarchical organization and prediction calibration within that setting. Future work includes scaling through self-supervised pretraining, extending the ontology to finer-grained tasks, and automating symbolic rule discovery via inductive logic programming (Muggleton and Raedt, 1994).

References

Miltiadis Allamanis, Earl T Barr, Premkumar Devanbu, and Charles Sutton. A survey of machine learning for big code and naturalness. *ACM Computing Surveys (CSUR)*, 51(4): 1–37, 2018a.

Miltiadis Allamanis, Marc Brockschmidt, and Mahmoud Khademi. Learning to represent programs with graphs. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018b.

Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. code2vec: Learning distributed representations of code. In *Proceedings of the ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 40:1–40:29, 2019.

Babelfish. Babelfish: Universal abstract syntax tree (uast) framework. Online documentation, version 2.x, 2017. URL https://docs.sourced.tech/babelfish/uast/. Accessed: 2025-12-15.

Artur d'Avila Garcez, Marco Gori, Luis C. Lamb, Luciano Serafini, Michael Spranger, and Son N. Tran. Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning. *Journal of Applied Logics*, 6(4):611–632, 2019.

Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1536–1547, 2020.

Octavian-Eugen Ganea, Gary Bécigneul, and Thomas Hofmann. Hyperbolic neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, pages 5350–5360, 2018.

Eleonora Giunchiglia, Mihaela Catalina Stoian, and Thomas Lukasiewicz. Deep learning with logical constraints. *arXiv preprint arXiv:2205.00523*, 2022.

Albert Gu, Frederic Sala, Beliz Gunel, and Christopher Ré. Learning mixed-curvature representations in product spaces. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.

Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. GraphCodeBERT: Pre-training code representations with data flow. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

Petr Hájek. *Metamathematics of Fuzzy Logic*. Kluwer Academic Publishers, 1998.

Vincent J Hellendoorn, Charles Sutton, Rishabh Singh, Petros Maniatis, and David Bieber. Global relational models of source code. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=B1lnbRNtwr.

Max Kochurov, Rasul Karimov, and Serge Kozlukov. Geoopt: Riemannian optimization in PyTorch, 2020. URL https://arxiv.org/abs/2005.02819.

Dmitri Krioukov, Fragkiskos Papadopoulos, Maksim Kitsak, Amin Vahdat, and Marián Boguñá. Hyperbolic geometry of complex networks. *Physical Review E*, 82(3):036106, 2010.

Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. Gated graph sequence neural networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.

Stephen Muggleton and Luc De Raedt. Inductive logic programming: Theory and methods. *Journal of Logic Programming*, 19:629–679, 1994.

Maximilian Nickel and Douwe Kiela. Poincaré embeddings for learning hierarchical representations. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 6338–6347, 2017.

Changan Niu, Chuanyi Li, Bin Luo, and Vincent Ng. Deep learning meets software engineering: A survey on pre-trained models of source code. *arXiv preprint arXiv:2205.11739*, 2022.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992, 2019.

Luciano Serafini and Artur d'Avila Garcez. Logic tensor networks: Deep learning and logical reasoning from data. In *Proceedings of the International Workshop on Neural-Symbolic Learning and Reasoning (NeSy)*, pages 1–13, 2016.

Emile Van Krieken, Erman Acar, and Frank Van Harmelen. Analyzing differentiable fuzzy logic operators. *Artificial Intelligence*, 302:103602, 2022.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.

Jingyi Xu, Zilu Zhang, Tal Friedman, Yitao Liang, and Guy Van den Broeck. A semantic loss function for deep learning with symbolic knowledge. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 5498–5507, 2018.

Appendix A. Domain exclusion pairs

The 14 mutually exclusive domain pairs in $\mathcal{R}_{\text{excl}}$ (co-occurrence $< 1\%$):

1. Mobile $\perp$ Game Development
2. Security $\perp$ Automation
3. DevOps $\perp$ GUI Development
4. System Programming $\perp$ Web Development
5. Database $\perp$ Game Development
6. Testing $\perp$ Security
7. Networking $\perp$ Scientific Computing
8. Data Science $\perp$ Mobile Development
9. ML/AI $\perp$ System Programming
10. GUI Development $\perp$ Networking
11. Automation $\perp$ Database
12. DevOps $\perp$ Game Development
13. Security $\perp$ GUI Development
14. Mobile $\perp$ Scientific Computing

Appendix B. Hyperparameter Sensitivity

Table 5 reports grid search results for symbolic loss weights and architectural choices. Each hyperparameter was tuned independently while keeping all others fixed at their optimal values.

Table 5: Hyperparameter sensitivity analysis.

Hyperparameter	Range	Optimal	Top-1 Impact
λ_{con}	$[0.1, 1.0]$	0.5	$\pm 2.3\%$
λ_{struct}	$[0.1, 0.5]$	0.3	$\pm 1.2\%$
λ_{behav}	$[0.1, 0.5]$	0.2	$\pm 0.8\%$
λ_{excl}	$[0.3, 0.7]$	0.5	$\pm 2.1\%$
GNN layers	$[2, 4]$	3	$\pm 3.5\%$
Embedding dim	$[128, 512]$	256	$\pm 1.9\%$

Training configuration. All models are trained with the Adam optimizer (β_1=0.9, β_2=0.999, weight decay 1×10^{-5}, learning rate 5×10^{-5}) with a batch size of 256, for up to 100 epochs with early stopping at patience 15, on a single NVIDIA A10G GPU. Riemannian gradient descent for hyperbolic parameters uses the `geoopt` library (Kochurov et al., 2020). The E3 hyperbolic variant incurs approximately $2\times$ the wall-clock time per epoch relative to the GNN-only baseline (E1) on the same hardware, measured as the mean of the final 10 training epochs.

Appendix C. Hyperbolic Curvature Sensitivity

Table 6 reports the effect of varying hyperbolic curvature c on zero-shot generalization performance. These results were obtained during the model-selection stage (single-seed tuning runs) and are used exclusively to justify the choice of c=1.0; they are not directly comparable to the multi-seed geometry ablation in Table 2. We evaluated $c \in \{0.1, 0.5, 1.0, 2.0, 5.0\}$ while keeping all other hyperparameters fixed. Results show that $c = 1.0$ achieves the best trade-off between in-distribution and zero-shot accuracy. Lower curvatures ($c < 1.0$) produce embeddings closer to Euclidean space, reducing the hierarchical inductive bias and degrading zero-shot Top-1 by up to 4.3pp. Higher curvatures ($c > 1.0$) overly concentrate embeddings near the ball's boundary, leading to numerical instability and training divergence in 2 out of 5 runs for $c = 5.0$.

Table 6: Effect of hyperbolic curvature c on performance. $c = 1.0$ balances in-distribution and zero-shot metrics.

Curvature c	In-Dist Top-1	ZS Top-1	ZS Top-5
0.1	36.21	15.87	48.32
0.5	35.44	18.73	50.12
1.0	34.49	20.16	50.85
2.0	32.88	19.52	49.73
5.0	28.91*	17.64*	46.21*

* Averaged over 3 stable runs; 2 runs diverged.

Table 7: Per-class F1 scores for the GNN-only classifier. Domains with <15 test samples achieve near-zero F1.

Domain	Support	Precision	F1
Testing	304	0.484	0.589
Science	260	0.570	0.665
Data Processing	186	0.467	0.445
Web	186	0.388	0.379
Scientific Computing	164	0.385	0.390
Systems Programming	141	0.318	0.203
Quantum Computing	97	0.607	0.272
DevOps†	55	0.591	0.338
Security†	23	0.500	0.080
Web Development	20	0.500	0.091
Networking	12	0.000	0.000
GUI	11	0.000	0.000
Machine Learning	9	0.000	0.000
Cloud Computing	5	1.000	0.333
Macro Avg	1473	0.415	0.270
Weighted Avg	1473	0.462	0.440

† Held-out domains for zero-shot evaluation.

Appendix D. Per-Class Performance Analysis

Table 7 provides a detailed breakdown of per-class performance for the structure-only GNN classifier. The results reveal severe class imbalance effects driven by the long-tailed distribution of test samples across domains. Domains with fewer than 15 test samples (Networking, GUI, Machine Learning, Cloud Computing) achieve near-zero F1 scores, while larger classes (Testing with 304 samples, Science with 260 samples) reach F1 scores above 0.5. Notably, the held-out zero-shot domains—Security (23 samples) and DevOps (55 samples)—achieve F1 scores of 0.080 and 0.338 respectively when evaluated in-distribution during ablation studies, confirming that the model struggles with minority classes regardless of domain semantics. This pattern is consistent with the weighted average F1 (0.440) being substantially higher than the macro average (0.270), indicating that performance is dominated by high-support classes.

Appendix E. Dataset Statistics

This appendix reports dataset composition and structural statistics for the 7,615-file UAST corpus. Figure 3 summarizes the domain distribution and domain-behavior compatibility matrix. Figure 4 shows the distribution of UAST graph sizes, with most files containing 50–200 nodes but a long tail extending to over 4,000 nodes in complex modules. Figure 5 reports library usage and node type statistics. The left panel displays the top-20 most frequently imported libraries, with Qiskit (6,402 files) dominating due to our quantum computing repository collection, followed by scientific computing libraries (NumPy, SciPy). The right panel breaks down the distribution of UAST node types, confirming that function calls (`call`) constitute 67.8% of all nodes—consistent with Python's functional programming patterns.



(a) Domain distribution **(b) Domain-behavior ontology**

Figure 3: Dataset composition and symbolic knowledge base. (a) Distribution of 7,615 UAST graphs across 14 domains. Security and DevOps are held out for zero-shot evaluation. (b) Manually curated domain-behavior compatibility matrix $\mathbf{M}$ used in $\mathcal{L}_{behav}$.

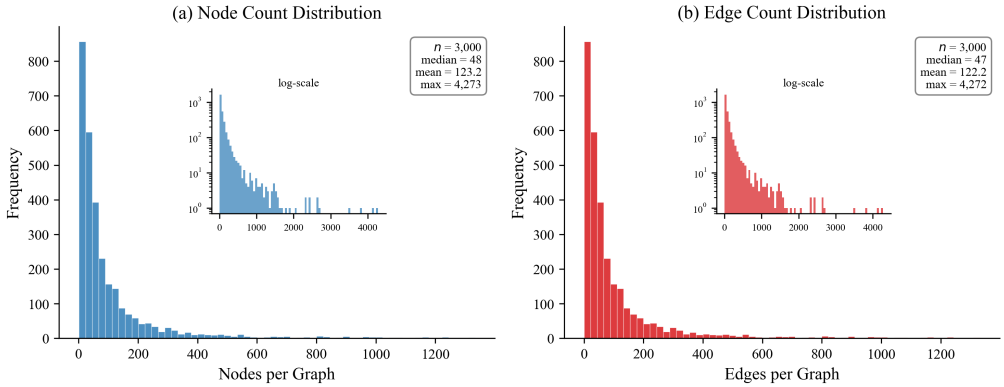


Figure 4: Distribution of node and edge counts across UAST graphs (median: 89 nodes, 87 edges). Inset shows log-scale view of the long tail extending to over 4,000 nodes.

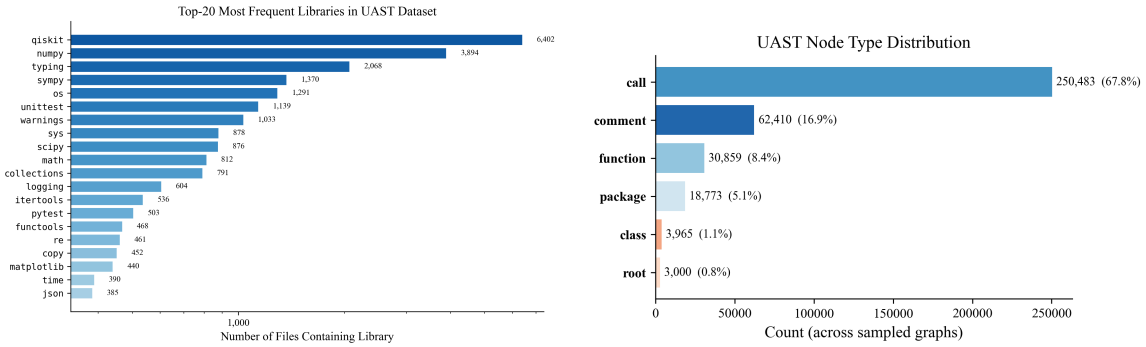


Figure 5: Dataset statistics. **Left:** Top-20 most frequently imported libraries, demonstrating domain coverage from quantum computing (Qiskit) to scientific computing (NumPy, SciPy) and testing frameworks. **Right:** UAST node type distribution. Function calls dominate (67.8%), followed by comments (16.9%) retained for structural analysis despite identifier stripping.

E.1. Symbolic Knowledge Base

Table 8 lists representative API-to-behavior mappings used to derive $\mathbf{s}_{behav}$. Each category is matched at two levels: import prefixes from `uast_package` nodes and call signatures from `uast_call` nodes. These rules are task-specific and were constructed via corpus inspection; the full rule set and `domain_ontology.json` will be released with the code upon publication.

Table 9 gives the exact binary entries of $\mathbf{M}_y \in \{0,1\}^{14\times 7}$. Figure 3b provides the visual overview; this table supplies the precise values needed for reproducibility.

E.2. UAST Dataset Samples

The Universal Abstract Syntax Tree (UAST) (Babelfish, 2017) is a language-agnostic intermediate representation that normalizes heterogeneous source languages into a unified node taxonomy, enabling structure-only analysis independent of surface-level identifiers. Representative examples illustrate the structure-preserving transformation applied during

Table 8: Representative API-to-behavior mappings used to derive $\mathbf{s}_{behav}$. Each behavioral flag is activated by matching import prefixes (`uast_package` nodes) or call signatures (`uast_call` nodes).

Behavior flag	Package prefixes (import)	Call signatures
`data_io`	csv, json, pandas, h5py, pickle	read_csv, load, dump, open
`math_compute`	numpy, scipy, sympy, math	dot, matmul, linalg, fft
`network_comm`	requests, flask, socket, urllib	get, post, connect, send
`visualization`	matplotlib, seaborn, plotly, PIL	plot, scatter, savefig, show
`model_training`	torch, sklearn, qiskit, tensorflow	fit, predict, backward, QuantumCircuit
`text_processing`	re, nltk, spacy, beautifulsoup	match, tokenize, encode, parse
`system_ops`	os, subprocess, threading, pathlib	mkdir, Popen, Thread, getenv

Table 9: Domain-behavior compatibility matrix $\mathbf{M}_y$ (exact binary entries for reproducibility). Column abbreviations: DI = data_io, MC = math_compute, NC = network_comm, VZ = visualization, MT = model_training, TP = text_processing, SO = system_ops. Figure 3b provides the visual overview.

Domain	DI	MC	NC	VZ	MT	TP	SO
Web Development	0	0	1	0	0	1	0
Data Science	1	1	0	1	0	1	0
ML/AI	1	1	0	1	1	0	0
Science	1	1	0	1	0	0	0
System	1	0	0	0	0	0	1
Database	1	0	0	0	0	1	0
Testing	0	0	0	0	0	0	1
Networking	0	0	1	0	0	0	1
Security	1	0	1	0	0	1	1
GUI	0	0	0	1	0	0	0
Game Dev	0	1	0	1	0	0	0
Cloud Computing	0	0	1	0	0	0	1
DevOps	0	0	1	0	0	0	1
Quantum Computing	0	1	0	1	1	0	0

preprocessing. Each example presents the original Python source alongside its UAST representation, demonstrating how syntactic structure is retained while implementation details are abstracted away.

E.2.1. Example 1: Single Function with Sequential Operations

A quantum circuit template function from the Qiskit library, demonstrating how sequential method calls are transformed into a structural graph.

Listing 1: Quantum circuit template from Qiskit repository.

```
from qiskit.circuit.quantumcircuit import QuantumCircuit

def template_nct_9d_3():
```

```
    qc = QuantumCircuit(3)
    qc.ccx(0, 1, 2)
    qc.cx(0, 2)
    qc.cx(2, 1)
    qc.x(2)
    qc.cx(1, 2)
    qc.ccx(0, 2, 1)
    qc.x(2)
    qc.cx(1, 2)
    qc.cx(2, 1)
    return qc
```

Listing 2: Structure-preserving UAST representation (identifiers stripped).


```
{
  "nodes": {
    "0":  {"id": 0,  "node_type": "uast_root",
           "children": [1, 2, "...", 12, 13]},
    "12": {"id": 12, "node_type": "uast_package",
           "code_snippet": "uast_package <MASKED>",
           "parents": [0]},
    "13": {"id": 13, "node_type": "uast_function",
           "code_snippet": "uast_function <MASKED>",
           "children": [14, 15, 16, 17, 18, 19, 20, 21, 22, 23],
           "parents": [0], "metadata": {"user_defined": true}},
    "14": {"id": 14, "node_type": "uast_call",
           "code_snippet": "uast_call <MASKED>", "parents": [13]},
    "15": {"id": 15, "node_type": "uast_call",
           "code_snippet": "uast_call <MASKED>", "parents": [13]},
    "16": {"id": 16, "node_type": "uast_call",
           "code_snippet": "uast_call <MASKED>", "parents": [13]},
    "17": {"id": 17, "node_type": "uast_call",
           "code_snippet": "uast_call <MASKED>", "parents": [13]},
    "18": {"id": 18, "node_type": "uast_call",
           "code_snippet": "uast_call <MASKED>", "parents": [13]},
    "19": {"id": 19, "node_type": "uast_call",
           "code_snippet": "uast_call <MASKED>", "parents": [13]},
    "20": {"id": 20, "node_type": "uast_call",
           "code_snippet": "uast_call <MASKED>", "parents": [13]},
    "21": {"id": 21, "node_type": "uast_call",
           "code_snippet": "uast_call <MASKED>", "parents": [13]},
    "22": {"id": 22, "node_type": "uast_call",
           "code_snippet": "uast_call <MASKED>", "parents": [13]},
    "23": {"id": 23, "node_type": "uast_call",
           "code_snippet": "uast_call <MASKED>", "parents": [13]}
  },
  "edges": [
    {"start_id": 0,  "end_id": 12, "directed_relation": "parent_node"},
    {"start_id": 0,  "end_id": 13, "directed_relation": "parent_node"},
    {"start_id": 13, "end_id": 14, "directed_relation": "parent_node"},
    {"start_id": 13, "end_id": 15, "directed_relation": "parent_node"}
  ]
}
```


The original function maps to node 13 of type `uast_function`. The 10 sequential gate calls become 10 child nodes (14–23), preserving call order through parent-child edges. Variable names, parameter values, and docstrings are stripped; the library import hides both the top-level package name and specific module path by replacing them with `<MASKED>`, enforcing identity-agnostic encoding.

E.2.2. Example 2: Multi-Function Module

A utility module with multiple functions and nested control flow from an HTTP library, illustrating how UAST preserves function call hierarchies and conditional logic structure.

Listing 3: HTTP multipart form encoding utilities.

```
import binascii
import codecs
import os
from io import BytesIO

def choose_boundary():
    boundary = binascii.hexlify(os.urandom(16))
    return boundary

def iter_field_objects(fields):
    if isinstance(fields, dict):
        for k, v in iter_fields(fields):
            yield (k, v)
    else:
        for k, v in fields:
            yield (k, v)

def iter_fields(fields):
    if hasattr(fields, 'items'):
        return fields.items()
    return fields

def encode_multipart_formdata(fields, boundary=None):
    if boundary is None:
        boundary = choose_boundary()
    body = BytesIO()
    for k, v in iter_field_objects(fields):
        body.write(b'--' + boundary + b'\r\n')
    return body.getvalue(), boundary
```

Listing 4: Structure-preserving UAST with nested function calls.

```
{
  "nodes": {
    "0":  {"id": 0,  "node_type": "uast_root",
           "children": [1, 2, "...", 43]},
    "1":  {"id": 1,  "node_type": "uast_package",
           "code_snippet": "uast_package <MASKED>"},
    "2":  {"id": 2,  "node_type": "uast_package",
           "code_snippet": "uast_package <MASKED>"},
    "5":  {"id": 5,  "node_type": "uast_function",
           "code_snippet": "uast_function <MASKED>",
           "children": [6, 7, 8, 9],
           "metadata": {"user_defined": true}},
    "10": {"id": 10, "node_type": "uast_function",
           "code_snippet": "uast_function <MASKED>",
           "children": [11, 12, 13, "...", 24],
           "metadata": {"user_defined": true}},
    "15": {"id": 15, "node_type": "uast_call",
           "code_snippet": "uast_call <MASKED>",
           "parents": [10, 14]},
    "16": {"id": 16, "node_type": "uast_call",
           "code_snippet": "uast_call <MASKED>",
           "parents": [14]}
  },
  "edges": [
    {"start_id": 0,  "end_id": 5,  "directed_relation": "parent_node"},
    {"start_id": 10, "end_id": 15, "directed_relation": "parent_node"},
    {"start_id": 14, "end_id": 15, "directed_relation": "parent_node"}
  ]
}
```

The four user-defined functions become separate `uast_function` nodes. Nested calls such as `iter_fields` inside `iter_field_objects` are represented through parent-child edges. Node 15 (`isinstance`) carries two parents (nodes 10 and 14), encoding the conditional branch structure without exposing variable names or string literals.

E.2.3. Example 3: Import-Heavy Module

A quantum state class with extensive library dependencies, illustrating UAST's handling of complex class hierarchies.

Listing 5: Density matrix class with extensive imports (abbreviated).

```
import warnings
from numbers import Number
import numpy as np
from qiskit.circuit.quantumcircuit import QuantumCircuit
from qiskit.circuit.instruction import Instruction
from qiskit.exceptions import QiskitError
from qiskit.quantum_info.states.quantum_state import QuantumState
from qiskit.quantum_info.operators.operator import Operator
# ... 7 more qiskit imports ...

class DensityMatrix(QuantumState):
    def __init__(self, data, dims=None):
        if isinstance(data, (list, np.ndarray)):
            self._data = np.asarray(data, dtype=complex)
        elif hasattr(data, 'to_operator'):
            op = data.to_operator()
            self._data = op.data

    def is_valid(self, atol=None, rtol=None):
        if not np.allclose(self.trace(), 1, rtol=rtol, atol=atol):
            return False
    # ... 23 more methods ...
```

Listing 6: Structure-preserving UAST of a class with extensive dependencies.

```
{
  "nodes": {
    "0":  {"id": 0,  "node_type": "uast_root",
           "children": [1, 2, "...", 61]},
    "13": {"id": 13, "node_type": "uast_package",
           "code_snippet": "uast_package <MASKED>"},
    "14": {"id": 14, "node_type": "uast_package",
           "code_snippet": "uast_package <MASKED>"},
    "15": {"id": 15, "node_type": "uast_package",
           "code_snippet": "uast_package <MASKED>"},
    "16": {"id": 16, "node_type": "uast_package",
           "code_snippet": "uast_package <MASKED>"},
    "26": {"id": 26, "node_type": "uast_package",
           "code_snippet": "uast_package <MASKED>"},
    "27": {"id": 27, "node_type": "uast_class",
           "code_snippet": "uast_class <MASKED>",
           "children": [28, 54, 58, "...", 219]}
  }
}
```

The 15 import statements become individual `uast_package` nodes; both top-level package names and specific submodule paths are fully masked to prevent label leakage, preserving only the structural fact that an import dependency exists. Multiple imports from the same package appear as separate nodes, preserving fine-grained dependency structure. The class `DensityMatrix` maps to node 27 with 25 child `uast_function` nodes representing its methods; method bodies are further decomposed into call graphs that encode API usage patterns without exposing argument values or type annotations.

Summary. Across all three examples, the UAST transformation preserves function definitions, call sequences, control flow branches, class hierarchies, and import dependencies, while removing variable names, parameter values, string and numeric literals, docstrings, and specific module paths. This design ensures that the GNN learns from algorithmic patterns and structural invariants rather than implementation-specific surface forms, enabling robust generalization across identifier obfuscation and code refactoring.

Appendix F. Hyperbolic Embedding Analysis

We provide a quantitative analysis of the geometric structure induced by hyperbolic vs. Euclidean retrieval, complementing the geometry ablation in Section 3.3. All figures are mean±std over 5 seeds; E2 (Euclidean retrieval) and E3 (hybrid hyperbolic) share the same Euclidean GAT encoder and are compared under the balanced stopping criterion.

Domain Hierarchy Encoding. We measure the Poincaré norm of each domain's mean embedding (E3) as a proxy for hierarchical depth—points near the boundary are more specialized, while points near the origin are more general. Specialized domains such as *Security* and *Scientific Computing* exhibit higher norms, while broader domains such as *Systems Programming* and *Data Processing* occupy more central positions. This radial structure is confirmed by a Spearman correlation of $\rho = -0.253 \pm 0.054$ ($p < 0.05$, 5-seed mean) between domain norm and the number of active behavioral categories: domains with broader behavioral profiles are geometrically closer to the origin, consistent with the hyperbolic hierarchy hypothesis. The Euclidean retrieval space (E2) does not exhibit a meaningful norm-depth correlation (undefined: unit-sphere normalization forces constant embedding norms).

Class Separation. Table 10 reports geometry diagnostics for E1–E4. E3 achieves a separation ratio of 0.803±0.065 versus 0.680±0.028 for E2, a statistically significant improvement (p=0.020, Cohen's d=1.69). Silhouette scores are negative for all variants, as expected given the 14-class long-tail distribution.

Table 10: Embedding geometry diagnostics (mean±std, 5 seeds, balanced stopping). Sep. Ratio = $d_{\text{inter}}/d_{\text{intra}}$.

Model	Intra-class $\bar{d}$	Inter-class $\bar{d}$	Sep. Ratio	Silhouette
E1: CE-only	8.613±0.203	5.664±0.292	0.657±0.027	-0.184 ± 0.045
E2: Eucl. Contr.	0.767±0.020	0.527±0.040	0.680±0.028	-0.054 ± 0.009
E3: Hybrid Hyp.	0.152±0.048	0.124±0.030	**0.803±0.065**	-0.150 ± 0.047
E4: Fully Hyp.	0.163±0.021	0.127±0.013	0.783±0.047	-0.180 ± 0.030

Retrieval Metrics. Table 11 reports MAP@5 and kNN-10 for E1–E4. E3 achieves MAP@5 of 0.303±0.011 vs. 0.301±0.024 for E2 (not statistically significant, p=0.873); one notable difference is reduced seed variance in MAP@5 for E3 (std 0.011 vs. 0.024 for E2), suggesting that hyperbolic retrieval may produce more consistent zero-shot behavior across training seeds, though kNN-10 slightly favors E2 (0.471 vs. 0.462).

Table 11: Retrieval metrics (mean±std, 5 seeds, balanced stopping).

Model	MAP@5	MAP@5 std	kNN-10 Acc.
E1: CE-only	0.000	0.000	0.000
E2: Eucl. Contr.	0.301	0.024	0.471±0.020
E3: Hybrid Hyp.	**0.303**	**0.011**	0.462±0.024
E4: Fully Hyp.	0.223	0.057	0.408±0.012

Appendix G. Stopping Criterion Sensitivity

The geometry ablation (Section 3.3) uses a balanced early-stopping criterion $\left(\frac{2\cdot\text{InDist}\cdot\text{ZS}}{\text{InDist}+\text{ZS}}\right)$ as the primary comparison, because E2–E3 targets zero-shot retrieval and accuracy-only stopping creates a criterion misalignment that disadvantages E3. Table 12 reports E2 and E3 under both criteria.

Table 12: E2 vs. E3 under accuracy-only vs. balanced stopping (mean±std, 5 seeds; Diff = E3 − E2). $^*p<0.05$, $^{**}p<0.01$ (paired t-test).

Criterion		InDist Top-1	ZS Top-1	MAP@5	Sep. Ratio	Norm-depth ρ
Accuracy	E2	41.1±1.3%	16.7±3.9%	0.287±0.037	0.687±0.042	—
	E3	31.8±2.6%	14.6±2.1%	0.277±0.010	0.836±0.062	-0.225 ± 0.076
	Diff	−9.3%**	−2.1% n.s.	−0.010 n.s.	+0.149**	—
Balanced	E2	37.0±5.7%	17.9±2.5%	0.301±0.024	0.680±0.028	—
	E3	30.4±2.1%	19.0±1.8%	0.303±0.011	0.803±0.065	-0.253 ± 0.054
	Diff	−6.6%*	+1.2% n.s.	+0.002 n.s.	+0.123*	—

Under accuracy-only stopping, E3 ZS Top-1 (14.6%) falls below E2 (16.7%), while under balanced stopping E3 slightly exceeds E2 (19.0% vs. 17.9%). The reversal—−2.1% to +1.2%—indicates that accuracy-only stopping selects checkpoints that maximize in-distribution accuracy at the cost of zero-shot readiness, penalizing E3's hierarchical regularization. Critically, the Sep. Ratio advantage of E3 over E2 is preserved under both criteria (p=0.002 accuracy-only; p=0.020 balanced), confirming that the geometric claim is not an artifact of criterion choice.

Appendix H. Full Obfuscation Robustness Results

Table 13 extends Table 1 to all five models and five obfuscation conditions (mean±std, 5 seeds). Obfuscation protocol: C0 = clean, C1 = import identifiers masked only, C2 = in-function identifiers stripped only, C3 = 50% partial obfuscation, C4 = all identifiers fully stripped. All models trained on C0, tested on each condition independently.

Input fairness note. Because the processed dataset does not retain raw `.py` source files, pretrained baselines (GraphCodeBERT, CodeBERT, TF-IDF) were evaluated on code-like text reconstructed from UAST metadata: import statements, comments, and function/class name tokens. Identical obfuscation transforms were applied to this reconstructed text for all surface-form models. This protocol is consistent across all text-based baselines but

differs from fine-tuning on full raw source code; absolute clean accuracy may therefore be somewhat lower than reported on standard code benchmarks.

SymAlign-UAST interpretation note. SymAlign-UAST in this robustness benchmark re-uses the E3 geometry-ablation variant trained under the accuracy-based validation criterion, without a dedicated supervised classification head for this task. Its role here is as a *structure-family representative* demonstrating invariance to identifier corruption, not as a best-accuracy model. The lower clean Top-1 (31.91%) relative to GNN-only (47.79%) is consistent with the geometry trade-off characterized in Section 3.3.

Table 13: Top-1 accuracy (%) across all obfuscation conditions (mean±std, 5 seeds). Retention = C4/C0.

Model	C0 (Clean)	C1 (Imp. mask)	C2 (ID strip)	C3 (50%)	C4 (Full)	Ret.
GraphCodeBERT	96.59±0.41	19.21±3.58	52.72±0.89	33.85±10.00	13.89±1.33	0.144
CodeBERT	96.21±0.22	24.11±5.91	52.78±0.78	37.87±6.56	20.49±6.41	0.213
TF-IDF + LR	93.62±0.00	12.63±0.00	93.62±0.00	56.97±12.89	12.63±0.00	0.135
GNN-only	47.79±0.00	47.79±0.00	47.79±0.00	47.79±0.00	47.79±0.00	1.000
SymAlign-UAST	31.91±2.47	31.91±2.47	31.91±2.47	31.91±2.47	31.91±2.47	1.000

Notable anomaly: TF-IDF under C2. TF-IDF accuracy under in-function identifier stripping (C2: 93.62%) is identical to its clean accuracy (C0), while import-masking only (C1: 12.63%) causes catastrophic collapse—identical to full obfuscation (C4). This confirms that TF-IDF classification signal derives exclusively from import-path tokens; removing only in-function identifiers has no effect, while masking imports alone destroys performance entirely.

C1 vs. C2 vulnerability decomposition. The asymmetry between C1 and C2 provides an interpretable decomposition of surface-form model vulnerability. GraphCodeBERT drops from 96.59% to 19.21% under C1 (import masking, −80.1%) but only to 52.72% under C2 (identifier stripping, −45.4%), indicating that import tokens are the single most important signal, but broader identifier context contributes additionally. CodeBERT shows the same pattern (C1: −74.9%; C2: −45.1%).

CodeBERT vs. GraphCodeBERT. CodeBERT retains a slightly higher fraction of clean accuracy under full obfuscation (21.3% vs. 14.4%), but both retain far less than structural models. The large CodeBERT seed variance under C4 (±6.41%) suggests inconsistent partial recovery of structural signal across seeds.

Appendix I. Symbolic Regularization Ablation (S0–S6)

This appendix reports the full results for the symbolic regularization analysis described in Section 3.4, including the shuffled and noisy ontology controls (S4, S5) and single-rule ablations (S6). All variants use the E3 Hybrid Hyperbolic backbone, 3 seeds (42/123/456), $\lambda_{\text{logic}} = 0.3$, and early stopping patience 10.

S4 and S5 controls (Table 14). Shuffled-ontology (S4) and 20%-noisy-ontology (S5) perform comparably to S1 across all reliability metrics: ViolRate 0.285 and 0.286 vs. 0.280 (S1); ECE 0.262 and 0.255 vs. 0.264. Neither gap exceeds the pre-registered threshold, reinforcing the main text's finding that, at the current dataset scale, aggregate regularization effects are not clearly distinguishable by ontology quality. S5 shows a slightly more negative EntCorr (-0.386 vs. -0.360), suggesting that mild noise may paradoxically encourage the model to rely on uncertainty signals more, though this difference is also not statistically significant.

S6 single-rule ablations (Table 14). No individual rule (S6_struct, S6_behav, S6_excl) outperforms S1 full. S6_behav achieves the lowest ViolRate (0.279) and most negative EntCorr (-0.377), but the three constraints are complementary—no single rule dominates.

Table 14: Full symbolic regularization results: controls (S4, S5) and single-rule ablations (S6). Mean over 3 seeds (42, 123, 456). S1 (current method) repeated for reference.

Variant	Top-1	ECE	ViolRate	EntCorr	ZS Top-1
S1 Current-Sym	35.64%	0.2640	0.2799	-0.360	19.19%
S4 Shuffled-Ontol.	35.46%	0.2623	0.2854	-0.345	18.92%
S5 Noisy-20%	34.89%	0.2548	0.2861	-0.386	19.14%
S6_struct only	35.62%	0.2640	0.2828	-0.354	19.01%
S6_behav only	35.37%	0.2608	0.2785	-0.377	18.62%
S6_excl only	35.21%	0.2586	0.2820	-0.366	18.92%

Rule-type breakdown. Across all variants, structural violation rate is approximately 0.0009 and exclusion violation rate is 0, so total violation rate is dominated by behavioral violations (mean ≈ 0.84 of total). The behavioral rule is the operationally active constraint at the current scale; structural and exclusion rules contribute negligible direct violation signal, though they may provide indirect regularization effects on the loss landscape.

Appendix J. Component Ablation

Table 15 reports single-seed results for six loss configurations (A1–A6), with multi-seed robustness established in Tables 2 and 4.

Table 15: Component ablation: loss configurations A1–A6 (single exploratory seed).

Config	$\mathcal{L}_{con}$	$\mathcal{L}_{struct}$	$\mathcal{L}_{behav}$	$\mathcal{L}_{excl}$	Top-1	MRR	ZS Top-1	ZS Top-10
A1: $\mathcal{L}_{CE}$ only					**40.60**	**0.5770**	0.34	72.41
A2: $+\mathcal{L}_{con}$	✓				32.32	0.5013	**20.16**	87.98
A3: $+\mathcal{L}_{struct}$	✓	✓			31.36	0.4944	19.14	<u>89.23</u>
A4: $+\mathcal{L}_{behav}$	✓		✓		17.65	0.2879	<u>20.98</u>	85.34
A5: $+\mathcal{L}_{excl}$	✓			✓	31.98	0.4956	15.21	**90.56**
A6: Full SymAlign	✓	✓	✓	✓	<u>33.88</u>	<u>0.5184</u>	17.31	88.91